

Riduzione del rumore in audio musicale senza dati di addestramento puliti

Lorenzo Aghilar
Politecnico di Torino

s334086@studenti.polito.it

Francesco Di Santo
Politecnico di Torino

s331336@studenti.polito.it

Giovanna Rosace
Politecnico di Torino

s332938@studenti.polito.it

Abstract

Il progetto tratta la riduzione del rumore (denoising) di un segnale audio musicale e l'analisi della rete neurale corrispondente per la realizzazione dell'obiettivo. Il problema di speech denoising è stato precedentemente trattato considerando segnali puliti come target durante l'addestramento. Tuttavia, un approccio più originale prevede l'utilizzo di segnali unicamente rumorosi, sia in ingresso che come target. L'obiettivo è quello di fornire una dimostrazione pratica e un'analisi più approfondita sull'addestramento di una rete neurale usando solo campioni audio rumorosi, nel contesto musicale anziché quello discorsivo. Il lavoro proposto dimostra che è possibile questo tipo di allenamento con un architettura chiamata Deep Complex U-Net con 20 layer.

1. Introduzione

L'utilizzo del Deep Learning[1] in ambito audio è senza dubbio diventato molto popolare e, nel corso degli anni, sono state progettate nuove tecniche con l'obiettivo di manipolare i segnali e lavorarci con specifici obiettivi. Il punto di forza di questa metodologia deriva dalla possibilità di creare relazioni e pattern tra una serie numerosa di dati, utilizzando un algoritmo di Machine Learning su uno specifico modello. Un esempio rilevante, su cui si basa questo lavoro, è la riduzione del rumore di un segnale audio discorsivo (speech denoising), presentato da Kashyap et al[2], che a sua volta estende il lavoro di Lhtinen et al[3], dove viene trattato il denoise delle immagini attraverso l'utilizzo di immagini rumorose, senza l'impiego di immagini pulite. In [2] l'autore utilizza unicamente segnali audio con rumore per allenare la rete, al contrario di altre trattazioni che prevedono l'uso di segnali audio puliti.

Il nostro lavoro ha come obiettivo quello di proporre una trattazione musicale, con metodologia Noise2Noise, basandoci sui lavori appena citati. Il codice sorgente del nostro modello di denoiser musicale N2N è disponibile su GitHub¹.

2. Teoria e background

2.1. Condizioni necessarie per il Noise2Noise

Come sostenuto dagli autori di [3] e [2], il denoise di immagini o audio con campioni rumorosi è possibile se le seguenti condizioni sono soddisfatte.

- Il rumore aggiunto al segnale di input e al segnale target hanno media nulla.
- La correlazione tra il rumore in input e il rumore in output tende a un valore vicino a zero.

La prima condizione assicura che il valore medio del target sia lo stesso del valore medio dell'input. La seconda condizione assicura che la rete non venga addestrata in modo da mappare le relazioni tra i rumori, ma che venga addestrata per rimuoverlo.

2.2. Short Time Fourier Transform e spettrogramma del segnale

Come fatto da [2], la rete neurale lavora sullo spettrogramma del segnale, impiegando la tecnica della STFT (Short Time Fourier Transform) [4]. Lo spettrogramma è una funzione $X_{t,f} : \mathbb{R}^2 \rightarrow \mathbb{C}$, (x, y) che associa le coordinate tempo-frequenza a un valore complesso, chiamato spettro tempo-frequenza. Dunque da un segnale, con la STFT è possibile trovare una matrice di numeri complessi con tempo e frequenza rappresentate rispettivamente da indici di colonna e di riga. La STFT è utilizzata per i segnali audio, in quanto non sono segnali costanti o periodici. La trasformazione prevede l'impiego della Trasformata di Fourier ogni R secondi (hop size) in una finestra temporale, definita dal parametro N , dove $R \leq N$. Questa disequazione presuppone che le finestre in cui avviene la trasformazione si possano sovrapporre. Tipicamente si parla di sovrapposizione riferendosi alla finestra di tempo comune tra una trasformata e quella subito successiva, calcolata come $\frac{N-R}{N} * 100$. Più nello specifico la STFT discreta è ottenuta nel seguente modo:

$$X[m, k] = \sum_{n=-\infty}^{+\infty} x(n)w(n - mR)e^{-i\frac{2\pi k}{N}n} \quad (1)$$

dove:

- m : istante di tempo
- R : hop size
- N : window size
- k : indice di frequenza ($0 \leq k \leq bins \leq N - 1$)

Inoltre w è una funzione porta definita come segue:

$$w(n - mR) = \begin{cases} 1, & \text{se } mR < n < N + mR \\ 0, & \text{altrimenti} \end{cases}$$

Infine, la ISTFT (Inverse Short Time Fourier Transform) è definita come segue:

$$x[n] = \frac{1}{\sum_j w^2(n - jR)} \sum_{m=0}^{M-1} \left[\frac{1}{N} \sum_{k=0}^{N-1} X[m, k] e^{i \frac{2\pi k}{N} (n - mR)} \right] \cdot w(n - mR) \quad (2)$$

3. Dataset

Data l'assenza, in letteratura, di dataset musicali che includano componenti rumorose sia nei segnali di input sia in quelli di target, per l'addestramento della rete neurale e le successive valutazioni sperimentali è stato costruito un dataset specifico a partire da due risorse preesistenti: *MUSDB18* [10] e *UrbanSound8K* [9].

Il primo comprende 150 tracce musicali stereofoniche appartenenti a generi differenti, suddivise in 100 tracce destinate alla fase di training e 50 alla fase di test. Il secondo, invece, contiene 1302 registrazioni di eventi sonori realistici, organizzate in 10 classi distinte: condizionatore d'aria, clacson, voci di bambini in attività ludiche, abbaiare canino, trapano, motore al minimo, sparo, martello pneumatico, sirena d'emergenza e musica di strada.

Il processo di generazione del dataset, ispirato all'approccio proposto da Kashyap et al. [2], è stato applicato sia al training set sia al test set. Nel primo caso, ogni coppia *input-target* è stata ottenuta selezionando casualmente una traccia musicale da *MUSDB18* e due rumori, N e M , appartenenti a classi differenti di *UrbanSound8K*. Il rumore di classe N è stato sovrapposto al segnale di input, mentre quello di classe M al segnale di target. Entrambi i rumori sono stati ciclati e normalizzati rispetto alla dinamica della traccia pulita, così da evitare configurazioni sbilanciate nelle quali la componente rumorosa potesse risultare dominante, o viceversa. Per il test set, invece, il segnale target corrisponde alla traccia audio originale priva di contaminazioni.

Seguendo questo approccio, sono state generate due tipologie principali di dataset:

- **Mixed dataset:** input e target contaminati con rumori appartenenti a classi differenti. In una sottoparte dei

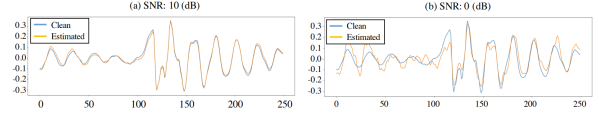


Figure 1. Denoise di un segnale ottenuto senza considerare la fase

casi, uno o entrambi i rumori sono stati sostituiti da rumore bianco gaussiano, così da incrementare la robustezza del modello.

- **Class-based datasets:** undici versioni distinte (una per ciascuna delle dieci classi di *UrbanSound8K*, più una versione con rumore bianco), ottenute mantenendo costante la classe di rumore sovrapposta al segnale input.

Da ogni coppia musica-rumore sono stati infine estratti segmenti di 3.74 s, selezionati in base alla combinazione ottimale di correlazione e diversità spettrale, così da garantire uniformità e qualità nei dati di addestramento ².

4. Metodologia

La rete utilizzata è la rete convolutiva Deep Complex U-Net con 20 layer (DCU-Net-20), impiegata da Choi ed al [5], che a sua volta è un'estensione della rete U-Net [6].

4.1. Spettrogrammi e Complex valued ratio mask

Come descritto nel paragrafo 2.2, la rete lavora con lo spettrogramma del segnale che può essere rappresentato come una matrice di numeri complessi. Dunque in ingresso è presente il segnale $x(t) \in \mathbb{R}$ che viene trasformato tramite STFT nel corrispettivo spettrogramma $X_{t,f} \in \mathbb{C}$. In uscita, invece, è presente lo spettrogramma $Y_{t,f} \in \mathbb{C}$, che verrà trasformato in $y(t) \in \mathbb{R}$ tramite ISTFT. L'idea è quella di utilizzare una rete che gestisca in modo efficace numeri complessi, così che possa lavorare sia sul modulo che sulla fase del segnale rumoroso. Una rete che gestisce solo numeri reali lavora solo con il modulo del segnale e ciò comporta la perdita della fase del segnale rumoroso. Questa metodologia può comunque essere impiegata per segnali poco rumorosi ($SNR > 10$), come illustrato nella figura 1.

L'approccio della rete è quello di ottimizzare una maschera cRM (complex valued ratio mask), da moltiplicare al segnale di partenza X . Ciò che viene stimato è dunque la matrice complessa $\hat{M}$, come illustrato nella figura 2. L'operazione che produce lo spettrogramma in output è dunque la seguente:

$$\hat{Y}_{t,f} = \hat{M}_{t,f} \cdot X_{t,f} = |\hat{M}_{t,f}| \cdot |X_{t,f}| \cdot e^{i(\theta_{\hat{M}_{t,f}} + \theta_{X_{t,f}})} \quad (3)$$

²Una lista completa dei dataset generati seguendo la metodologia descritta è disponibile pubblicamente: [Link list of datasets](#)

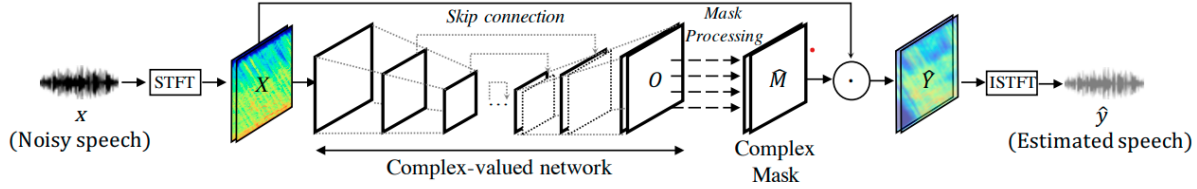


Figure 2. DCUnet-20

L'operazione svolta dalla rete è il calcolo delle coordinate polari della maschera, ovvero il calcolo di $\hat{M}_{t,f}^{mag} = |\hat{M}_{t,f}|$ e di $\hat{M}_{t,f}^{phase} = e^{i\theta_{\hat{M}_{t,f}}}$ nel seguente modo:

$$\hat{M}_{t,f}^{mag} = \tanh |O_{t,f}|, \quad \hat{M}_{t,f}^{phase} = O_{t,f} / |O_{t,f}|$$

dove $O_{t,f}$ è l'output della rete. Uno schema grafico è illustrato nella figura 3.

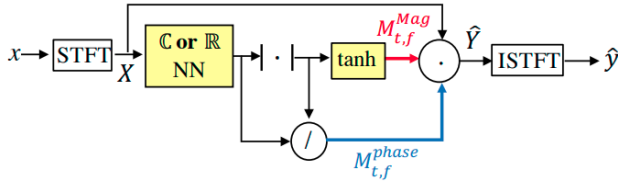


Figure 3. Architettura della rete con coordinate polari della maschera

4.2. Dettagli interni della DCUnet-20

Come già accennato, la rete in analisi è un'estensione della rete convolutiva U-Net. La struttura ha le seguenti caratteristiche:

- è di tipo encoder/decoder, dunque prevede una prima fase di contrazione, dove ad ogni layer si riducono le dimensioni delle feature map e aumentano il numero di canali e una seconda fase di espansione, dove le dimensioni delle feature map aumentano e il numero di canali diminuisce, fino a produrre la maschera complessa $\hat{M}$;
- prevede skip connections, grazie alle quali si evita il problema di exploding/vanishing gradients e si favorisce la propagazione delle feature map[8], che comporta la preservazione delle informazioni lungo la rete e, in particolare nella U-Net e DCUnet-20, la preservazione dei bordi[6];
- l'uso di layer convoluzionali complessi, che prevedono delle operazioni di convoluzione leggermente differenti;
- nella fase di encoding, l'operazione di max pooling, presente nella U-Net, viene sostituita da una convoluzione di tipo strided, sempre per una questione legata alla preservazione dell'informazione[5];

- la fase di decoding è realizzata tramite operazioni di deconvoluzione.

4.3. Convoluzione complessa

Come accennato nel paragrafo 4.2, i layer prevedono l'uso di blocchi complessi. Sia le feature map in input e in output, che i kernel, sono realizzati in modo che la metà dei canali rappresentino la parte reale e l'altra metà la parte immaginaria. Definiamo:

$$M_{in} = M_{in,R} + iM_{in,I} \quad (4)$$

$$K = K_R + iK_I \quad (5)$$

La feature map in output è definita, tramite convoluzione complessa, nel seguente modo:

$$M_{out} = M_{in} * K =$$

$$M_{in,R} * K_R + M_{in,R} * iK_I + iM_{in,I} * K_R + iM_{in,I} * iK_I = (M_{in,R} * K_R - M_{in,I} * K_I) + i(M_{in,R} * K_I + M_{in,I} * K_R) \quad (6)$$

Dunque, la feature map in uscita ha N_{out} canali, di cui $N_{out}/2$ che rappresentano la parte reale e $N_{out}/2$ che rappresentano la parte immaginaria:

$$\Re\{M_{out}\} = M_{in,R} * K_R - M_{in,I} * K_I \quad (7)$$

$$\Im\{M_{out}\} = M_{in,R} * K_I + M_{in,I} * K_R \quad (8)$$

4.4. Attivazione complessa

Una rete neurale che usa blocchi complessi ha una serie di funzioni di attivazioni che possono essere considerate. Definiamo z un numero complesso, prodotto da una convoluzione complessa. Le funzioni di attivazioni prese in esame da Trabelsi et al [7] sono la modReLU e la CReLU. Quella impiegata nella DCUnet-20 è la leaky CReLU, definita come segue:

$$\text{LeCReLU}(z) = \text{LeReLU}(\Re\{z\}) + i\text{LeReLU}(\Im\{z\})$$

La funzione $\text{LeReLU}(x)$ è definita nel seguente modo:

$$\text{LeReLU}(x) = \begin{cases} x, & \text{se } x > 0 \\ \alpha x, & \text{altrimenti} \end{cases} \quad (9)$$

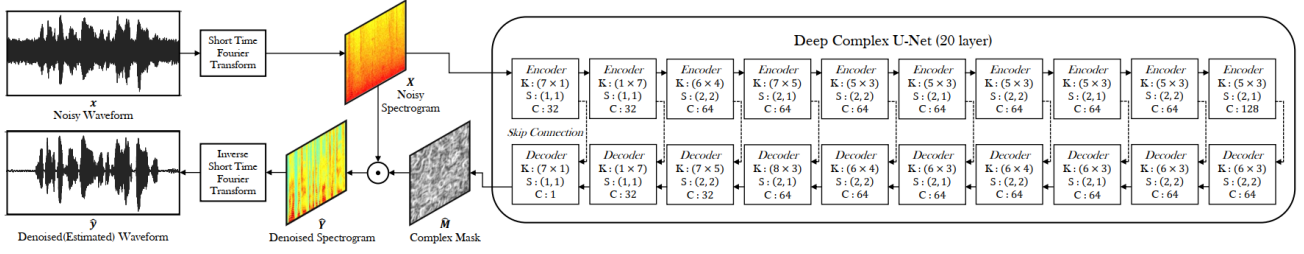


Figure 4. Rappresentazione più dettagliata della DCU-Net-20

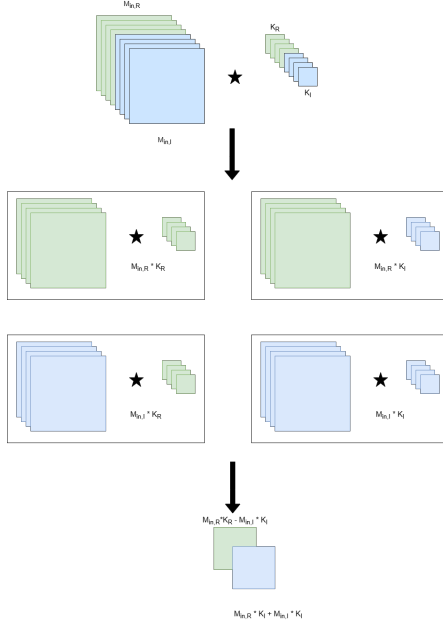


Figure 5. Convoluzione complessa

dove $\alpha = 0.01$ tipicamente.

Un'analisi interessante riguarda l'impiego di funzioni di attivazione olomorfe (o analitiche), ovvero differenziabili in senso complesso (vedere Appendice A). Normalmente, durante la backpropagation, per ogni attivazione è necessario calcolare 4 derivate per ogni coppia input/output. Se la funzione è olomorfa, è necessario calcolarne solo 2, dimezzando così di fatto la complessità computazionale[7]. La $\mathbb{C}\text{ReLU}$ ad esempio è olomorfa per $\theta_z \in]0, \pi[\vee \theta_z \in]\pi, 3\pi/2[$, dove θ_z è la fase di z . Per ulteriori approfondimenti, vedere l'Appendice B.

4.5. Inizializzazione dei pesi complessi

Un peso complesso della rete ha la seguente forma:

$$w = |w|e^{i\theta} = \Re\{w\} + \Im\{w\} \quad (10)$$

dove $|w|$ è il modulo e θ è la fase. Un'adeguata inizializzazione dei pesi è necessaria per evitare i fenomeni di vanishing/exploding gradients. L'approccio utilizzato è quello di Glorot e Bengio [11], che assicura la stessa varianza in

input, in output e nei gradienti dei vari layer.

Consideriamo:

$$\begin{cases} \text{var}(w) = \mathbb{E}[ww^*] - (\mathbb{E}[w])^2 = \mathbb{E}[|w|^2] - (\mathbb{E}[w])^2 \\ \text{var}(|w|) = \mathbb{E}[|w|^2] - (\mathbb{E}[|w|])^2 \end{cases} \quad (11)$$

dove $\mathbb{E}[w] = 0$. Si ottiene la seguente equazione:

$$\text{var}(w) = \text{var}(|w|) + (\mathbb{E}[|w|])^2 \quad (12)$$

Per ottenere $\mathbb{E}[|w|^2]$ e $(\mathbb{E}[|w|])^2$, è possibile tenere conto del modulo di w come variabile casuale con distribuzione di Rayleigh [7], con la seguente funzione di densità:

$$f(\rho, \sigma) = \frac{\rho}{\sigma^2} e^{-\frac{\rho^2}{2\sigma^2}} \quad (13)$$

dove:

- $\rho = |w|$,
- σ , detto modo, indica la deviazione standard di $\Re\{w\}$ e $\Im\{w\}$.

La media e la varianza sono calcolate come segue:

$$\mathbb{E}[\rho] = \sigma \sqrt{\frac{\pi}{2}} \quad (14)$$

$$\text{var}(\rho) = \frac{4 - \pi}{2} \sigma^2 \quad (15)$$

Infine, riprendendo l'equazione 12, si ottiene:

$$\text{var}(w) = \frac{4 - \pi}{2} \sigma^2 + \left(\sigma \sqrt{\frac{\pi}{2}} \right)^2 = 2\sigma^2 \quad (16)$$

Per rispettare il criterio di Glorot, $\sigma = \frac{1}{\sqrt{n_{in} + n_{out}}}$. Ciò comporta le seguenti varianze nel campionamento iniziale dei pesi:

$$\text{var}(w) = \frac{2}{n_{in} + n_{out}} \quad (17)$$

$$\text{var}(\Re\{w\}) = \text{var}(\Im\{w\}) = \frac{1}{n_{in} + n_{out}} \quad (18)$$

4.6. Batch normalization complessa

Normalmente, nelle reti neurali la tecnica di batch normalization consente un addestramento più stabile e veloce.

Tuttavia quella trattazione riguarda le reti neurali con valori reali. Nella DCUnet-20, la batch normalization è stata riformulata come segue[7]. Definiamo z un valore complesso, prodotto di una convoluzione complessa. Nella batch normalization complessa, z viene considerato come un vettore bidimensionale, $\mathbf{z} = (\Re\{\mathbf{z}\}, \Im\{\mathbf{z}\})^T$. L'operazione di normalizzazione è la seguente:

$$\hat{\mathbf{z}} = \mathbf{V}^{-\frac{1}{2}} (\mathbf{z} - \mathbb{E}[\mathbf{z}]) \quad (19)$$

dove:

$$\mathbb{E}[\mathbf{z}] = \begin{pmatrix} \mathbb{E}[\Re\{\mathbf{z}\}] \\ \mathbb{E}[\Im\{\mathbf{z}\}] \end{pmatrix} = \begin{pmatrix} \frac{1}{m} \sum_{i=1}^m \Re\{\mathbf{z}\} \\ \frac{1}{m} \sum_{i=1}^m \Im\{\mathbf{z}\} \end{pmatrix} \quad (20)$$

$$\mathbf{V} = \begin{pmatrix} Cov(\Re\{\mathbf{z}\}, \Re\{\mathbf{z}\}) & Cov(\Re\{\mathbf{z}\}, \Im\{\mathbf{z}\}) \\ Cov(\Im\{\mathbf{z}\}, \Re\{\mathbf{z}\}) & Cov(\Im\{\mathbf{z}\}, \Im\{\mathbf{z}\}) \end{pmatrix} \quad (21)$$

$$Cov(\Re\{\mathbf{z}\}, \Re\{\mathbf{z}\}) = \frac{1}{m} \sum_{i=1}^m (\Re\{\mathbf{z}\} - \mathbb{E}[\Re\{\mathbf{z}\}])^2 \quad (22)$$

$$Cov(\Im\{\mathbf{z}\}, \Im\{\mathbf{z}\}) = \frac{1}{m} \sum_{i=1}^m (\Im\{\mathbf{z}\} - \mathbb{E}[\Im\{\mathbf{z}\}])^2 \quad (23)$$

$$Cov(\Re\{\mathbf{z}\}, \Im\{\mathbf{z}\}) = Cov(\Im\{\mathbf{z}\}, \Re\{\mathbf{z}\}) = \frac{1}{m} \sum_{i=1}^m (\Re\{\mathbf{z}\} - \mathbb{E}[\Re\{\mathbf{z}\}])(\Im\{\mathbf{z}\} - \mathbb{E}[\Im\{\mathbf{z}\}]) \quad (24)$$

Infine definiamo i parametri $\gamma \in \mathbb{R}^{2,2}$ e $\beta \in \mathbb{R}^2$, dove γ è inizializzato a $\begin{pmatrix} \frac{1}{\sqrt{2}} & 0 \\ 0 & \frac{1}{\sqrt{2}} \end{pmatrix}$ e β inizializzato a $(0 \ 0)$. La batch normalization complessa è, dunque, la seguente:

$$BN(\hat{\mathbf{z}}_i) = \gamma \hat{\mathbf{z}}_i + \beta \quad (25)$$

4.7. Funzione di Loss

La funzione di Loss utilizzata è la weighted-SDR loss[5][2]. Consideriamo x il segnale in entrata rumoroso, y il segnale target e $\hat{y}$ il segnale stimato dalla rete.

$$loss_w SDR(x, y, \hat{y}) = -\alpha \frac{\langle y, \hat{y} \rangle}{\|y\| \|\hat{y}\|} - (1-\alpha) \frac{\langle x - y, x - \hat{y} \rangle}{\|x - y\| \|x - \hat{y}\|} \quad (26)$$

dove $\alpha = \frac{\|y\|^2}{\|y\|^2 + \|x + y\|^2}$ è il rapporto di energia tra segnale target e il rumore.

4.8. Selezione degli iperparametri e pipeline di training

Gli iperparametri principali, tra cui learning rate, batch size, profondità di rete, numero di canali e parametri STFT (`n_fft`, `hop_length`), sono stati selezionati tramite ottimizzazione manuale sulle curve di validazione. Il learning rate è gestito tramite scheduler OneCycleLR, che massimizza la velocità di convergenza riducendo lo stallo tra un

ciclo e l'altro. Il batch size è stato scelto compatibilmente con la memoria GPU, privilegiando il valore massimo che non comportasse errore di OOM (Out Of Memory).

La scelta del numero di layer (20) e la profondità dei canali (da 32 a 128) deriva dalla letteratura e da test preliminari sulla capacità di generalizzazione; valori maggiori comportavano overfitting precoce, con un deterioramento significativo delle metriche.

4.9. Aspetti complessi del progetto

La sfida principale affrontata riguarda la gestione delle operazioni complesse in ambiente PyTorch, che non fornisce supporto nativo per tutte le operazioni convoluzionali e di normalizzazione su tensori complessi. Tali operazioni sono state decomposte in sottoblocchi reali e immaginari sincronizzati. Uno degli altri aspetti critici è stato allineare temporalmente i tensori dopo le operazioni di STFT e ISTFT, specialmente nella fase di validazione, dove era necessario confrontare onde predette e target con precisione di singolo campione per un calcolo corretto delle metriche SNR.

4.10. Valutazione e metriche

La valutazione della qualità dell'algoritmo di denoising è effettuata sul test set calcolando per ogni campione il rapporto segnale-rumore (SNR) tra il segnale pulito di riferimento e il segnale predetto, definito come

$$SNR(\mathbf{x}_{\text{clean}}, \mathbf{x}_{\text{pred}}) = 10 \log_{10} \frac{\sum_{i=1}^N x_{\text{clean},i}^2}{\sum_{i=1}^N (x_{\text{clean},i} - x_{\text{pred},i})^2 + \epsilon} \quad (27)$$

dove $\mathbf{x}_{\text{clean}}$ e $\mathbf{x}_{\text{pred}}$ rappresentano rispettivamente i segnali pulito e predetto, mentre ϵ è un termine di stabilità numerica.

Il valore di riferimento è lo SNR medio del segnale predetto rispetto a quello pulito sul test set, indicato come *mean SNR predicted vs clean*. Viene inoltre riportato il miglioramento medio, *mean SNR improvement*, definito come:

$$\Delta SNR = SNR(\mathbf{x}_{\text{clean}}, \mathbf{x}_{\text{pred}}) - SNR(\mathbf{x}_{\text{clean}}, \mathbf{x}_{\text{noisy}}) \quad (28)$$

dove $\mathbf{x}_{\text{noisy}}$ è il segnale rumoroso in ingresso.

Inoltre, per una valutazione più robusta della qualità percettiva, viene calcolato il rapporto segnale-rumore segmentale (SSNR), che analizza il segnale a livello di frame temporali.

Analogamente all'SNR, viene riportato il *mean SSNR predicted vs clean* e il *mean SSNR improvement*, calcolato come differenza tra l'SSNR del segnale denoised e quello del segnale rumoroso.

Tali metriche forniscono una misura quantitativa dell'efficacia del modello nel migliorare la qualità del segnale audio rispetto al segnale rumoroso originale, con

l'SSNR che offre una valutazione più dettagliata delle prestazioni locali nel tempo.

5. Esperimenti

Il presente studio ha implementato e validato l'architettura DCUNet-20 per il denoising di segnali audio musicali, partendo dalla baseline di Kashyap et al. [2] e adattandola al nostro dataset. Si precisa che tutti gli esperimenti sono stati effettuati unicamente utilizzando il dataset creato con rumori bianchi gaussiani.

5.1. Esperimenti preliminari e identificazione dei problemi principali

Baseline (esperimenti 1-3): la configurazione iniziale seguiva esattamente i parametri del paper originale: 4 epoche di training, utilizzando il dataset di canzoni non segmentate. Per ogni brano, è stato effettuato un singolo taglio della parte iniziale, limitandosi ai primi 3.44 secondi, senza generare ulteriori segmenti o campioni. I risultati sono stati insoddisfacenti, con SNR di -14.593 ± 20.412 dB e loss -0.724 . Il raddoppio delle epoche non ha portato miglioramenti. L'adattamento dei parametri STFT per contenuti musicali ($n_fft=2048$, $hop_length=512$, $sample_rate=44100$) ha mostrato miglioramenti limitati, ma insufficienti.

Ottimizzazione hardware (esperimenti 4-6): l'aumento della lunghezza dei segmenti ed anche l'incremento del `batch_size`, ha causato problemi di memoria GPU. La riduzione della `batch_size` a 1 per stabilizzare il training ha prodotto SNR -1.953 ± 17.520 dB. L'aumento a `batch_size=4` con architettura più complessa (64) ha peggiorato i risultati (SNR -2.343 ± 18.509 dB), suggerendo overfitting.

5.2. Ottimizzazione learning rate e architettura

Scheduler e complessità (esperimenti 7-10): la modifica dello StepLR con $lr=0.005$ ha stabilizzato il training (SNR -2.202 ± 18.563 dB). Il ritorno della complessità del modello a valori più bassi (45) ha migliorato significativamente le prestazioni (SNR -1.38 ± 17.942 dB). Un ulteriore ritorno della complessità a 52 per 10 epoche ha raggiunto il miglior risultato preliminare di SNR -1.149 ± 16.289 dB solo alla nona epoca. Riportando la complessità a 45 e mantenendo 10 epoche, si è raggiunto SNR -0.339 ± 16.642 dB e loss -0.670 , confermando che una complessità moderata produce risultati migliori.

5.3. Breakthrough: correzioni algoritmiche

Esperimento 11: l'implementazione di modifiche cruciali ha prodotto il primo miglioramento evidente:

- Preprocessing: taglio dei segmenti 6 secondi dopo l'inizio della canzone per ridurre pause di silenzio,

- Correzione del bug nella Glorot initialization,
- Gradient clipping (`max_norm=1`) con $lr=0.001$,
- Scheduler ReduceLROnPlateau ottimizzato per 30 epoche.

Risultati: SNR iniziale 6.933 ± 7.987 dB, stabilizzato a 6.921 ± 5.604 dB (loss -0.583), rappresentando un miglioramento di oltre 20 dB rispetto alla baseline.

5.4. Ottimizzazione avanzata del dataset

Segmentazione avanzata (esperimento 12): aumento a 3.7 secondi con generazione di segmenti multipli per canzone (max 10000 train, 1000 test), allenando per 3 epoche ha prodotto SNR 4.49 dB.

Filtraggio intelligente (esperimento 13): si è implementato un filtraggio dei segmenti del dataset, basato su correlazione e diversità spettrale tra input e target, per selezionare solo i segmenti più informativi. La loss è stata modificata combinando SDR loss (80%) e spectral loss (20%) per bilanciare accuratezza temporale e fedeltà spettrale. Il training ha mostrato miglioramento graduale da 3.373 a 3.489 dB.

5.5. Correzione architetturale e cambio di metrica

Esperimento 14: è stata identificata e rettificata una discrepanza significativa tra l'implementazione dell'architettura e il paper originale. La complessità è stata impostata a 32, le skip connections sono state corrette, ed è stata adottata una MSE loss più semplice. E' stata adottata la metrica *mean SNR improvement* invece dell'SNR assoluto per una valutazione più robusta dell'efficacia del denoising.

Risultati: mean SNR improvement 3.96 dB (best: 9.20 dB, worst: 0.12 dB).

5.6. Configurazione finale ottimizzata

Esperimenti 15-25: ottimizzazione sistematica attraverso variazioni di `batch_size` (1-2-4), learning rate (tra $1e-6$ e $5e-4$), diversi scheduler (StepLR, ReduceLROnPlateau, CosineAnnealingLR, OneCycleLR), gradient clipping (0.1-1.0), ottimizzatori Adam/AdamW e diverse loss.

Configurazione finale (esperimento 25):

- **Ottimizzatore:** AdamW ($lr=5e-5$, $weight_decay=1e-4$)
- **Scheduler:** OneCycleLR ($max_lr=1.5e-4$, $pct_start=0.15$, $div_factor=4$, $final_div_factor=20$)
- **Architettura:** Complessità 32 con skip connections corrette
- **Dataset:** Segmenti 3.7s filtrati con preprocessing ottimizzato
- **Loss:** wSDR (weighted SDR) loss

Risultati finali: mean SNR improvement = +7.54 dB, Loss = -0.9698

5.7. Analisi dei risultati

Il percorso sperimentale ha evidenziato che:

- le configurazioni per speech non sono trasferibili direttamente al dominio musicale,
- la gestione dei silenzi e la segmentazione intelligente sono fondamentali,
- bug fixes e correzioni architetturali hanno rappresentato il breakthrough principale,
- l'uso di scheduler avanzati e regolarizzazione ha permesso il raggiungimento della configurazione ottimale con +7.54 dB di miglioramento SNR medio.

5.8. Risultati finali

I risultati confermano che è possibile ottenere prestazioni competitive utilizzando esclusivamente campioni rumorosi durante l'addestramento, aprendo nuove possibilità per scenari in cui dati puliti non sono disponibili.

I risultati ottenuti dalla configurazione finale per tutti e 12 i dataset di rumore, sono stati riportati nella tabella 1. Viene confrontato il modello allenato per lo speech denoising [2] ($N2N_{speech}$) a quello allenato da noi specializzato sulla musica ($N2N_{music}$).

Campioni audio di segmenti elaborati dal nostro modello sono disponibili nel repository GitHub³. La Figura 6 mostra la forma d'onda di un campione di test: nella prima traccia è riportato il segnale di riferimento (clean target), nella seconda lo stesso campione contaminato da rumore (noisy input) e, infine, nella terza il segnale stimato dal nostro modello (denoised output).

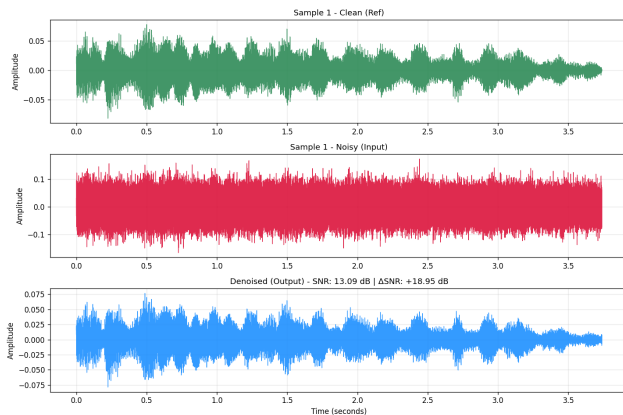


Figure 6. Confronto della forma d'onda di un campione: segnale pulito, segnale contaminato dal rumore bianco e segnale ricostruito dal modello.

³https://github.com/Frads01/NoiseRemover/tree/main/samples_e_valutazioni

6. Conclusioni

Questo lavoro ha dimostrato con successo la fattibilità dell'applicazione dell'approccio Noise2Noise al dominio musicale, raggiungendo risultati significativamente superiori rispetto alle architetture progettate per il speech denoising.

6.1. Apprendimenti chiave

La trasferibilità diretta di configurazioni ottimizzate per lo speech, al dominio musicale, si è rivelata limitata. I segnali musicali presentano caratteristiche spettrali e dinamiche temporali sostanzialmente diverse da quelli vocali, richiedendo un ripensamento completo dei parametri STFT ($n_{fft}=3072$, $hop_length=768$ per $sample_rate=48000$) e delle strategie di preprocessing.

6.2. Analisi delle prestazioni per categoria

I risultati mostrano prestazioni eccellenti con il rumore bianco (+7.67 dB), mentre le prestazioni con rumori realistici sono più variabili a causa di similarità timbriche e proprietà non-stazionarie dei rumori. L'SSNR occasionalmente supera l'SNR globale, indicando eccellenza nel miglioramento locale, ma possibili distorsioni globali.

6.3. Limitazioni e sfide affrontate

Le principali sfide tecniche hanno riguardato:

- Sincronizzazione temporale: l'allineamento preciso dei tensori dopo operazioni STFT/ISTFT per calcoli accurati delle metriche.
- Overfitting: architetture troppo complesse hanno mostrato degradazione precoce, richiedendo un bilanciamento attento tra capacità del modello e generalizzazione.

6.4. Estensioni future

I risultati aprono diverse direzioni promettenti, quali la validazione dell'approccio su generi musicali specifici (classica, jazz, elettronica), l'ottimizzazione per applicazioni real-time e l'esplorazione di varianti completamente auto-supervisionate che non richiedano nemmeno coppie di segnali rumorosi correlati.

6.5. Considerazioni finali

Questo lavoro dimostra che l'adattamento dell'approccio Noise2Noise al dominio musicale non solo è fattibile, ma può raggiungere prestazioni competitive senza la necessità di dati di training puliti. Il successo dell'implementazione conferma la validità teorica dell'approccio e apre nuove possibilità per scenari applicativi dove l'acquisizione di segnali musicali puliti è impraticabile o costosa.

Table 1. Denoising performance

Categoria del rumore	Modello	SNR [dB]	SNR improvement [dB]	SSNR [dB]	SSNR improvement [dB]
Rumore bianco	$N2N_{speech}$	10.52	-0.30	9.67	0.89
	$N2N_{music}$	18.49	7.67	16.95	8.18
Mixed	$N2N_{speech}$	0.76	-11.96	0.94	-10.55
	$N2N_{music}$	17.57	4.86	17.20	5.71
Aria condizionata (0)	$N2N_{speech}$	2.06	-12.00	2.00	-10.29
	$N2N_{music}$	14.75	0.70	13.37	1.08
Clacson (1)	$N2N_{speech}$	1.51	-16.40	1.32	-16.01
	$N2N_{music}$	16.82	-1.09	16.35	-0.98
Bambini che giocano (2)	$N2N_{speech}$	0.54	-14.41	0.77	-14.69
	$N2N_{music}$	12.80	-2.15	12.86	-2.18
Cane abbaia (3)	$N2N_{speech}$	1.59	-10.04	1.87	-16.54
	$N2N_{music}$	13.64	2.01	16.69	-1.71
Trivellazione (4)	$N2N_{speech}$	1.81	-11.97	1.68	-11.48
	$N2N_{music}$	14.43	0.65	13.83	0.68
Motore al minimo (5)	$N2N_{speech}$	1.03	-11.26	1.01	-9.61
	$N2N_{music}$	13.40	1.11	12.08	1.46
Colpo di pistola (6)	$N2N_{speech}$	1.68	-8.56	2.36	-15.16
	$N2N_{music}$	13.10	2.85	15.89	-1.62
Martello pneumatico (7)	$N2N_{speech}$	1.26	-14.15	1.05	-13.95
	$N2N_{music}$	16.25	0.84	15.57	0.57
Sirena (8)	$N2N_{speech}$	2.25	-12.14	2.18	-11.88
	$N2N_{music}$	14.57	0.18	14.42	0.37
Musica di strada (9)	$N2N_{speech}$	0.52	-12.63	0.59	-12.27
	$N2N_{music}$	13.11	-0.05	12.85	-0.02

References

- [1] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning", Nature, vol. 521, no. 7553, pp. 436–444, May 2015, doi:10.1038/nature14539.
- [2] M. M. Kashyap, A. Tambwekar, K. Manohara, and S. Natarajan, "Speech Denoising Without Clean Training Data: A Noise2Noise Approach", Interspeech 2021, Aug. 2021, doi:10.21437/interspeech.2021-1130.
- [3] J. Lehtinen et al., "Noise2Noise: Learning Image Restoration without Clean Data", arXiv preprint arXiv:1803.04189, 2018.
- [4] J. Allen, "Short term spectral analysis, synthesis, and modification by discrete Fourier transform", IEEE Trans. Acoustics, Speech, Signal Processing, vol. 25, no. 3, pp.235–238, 1977, doi:10.1109/TASSP.1977.1162950.
- [5] H.-S. Choi et al., "Phase-aware Speech Enhancement with Deep Complex U-Net", arXiv preprint arXiv:1903.03107, 2019.
- [6] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation", arXiv preprint arXiv:1505.04597, 2015.
- [7] C. Trabelsi et al., "Deep Complex Networks", arXiv preprint arXiv:1705.09792, 2018.
- [8] Z. Lai et al., "Rethinking Skip Connections in Encoder-decoder Networks for Monocular Depth Estimation", arXiv preprint arXiv:2208.13441, 2022.
- [9] J. Salamon, C. Jacoby and J. P. Bello, "A Dataset and Taxonomy for Urban Sound Research", Proc. 22nd ACM Int. Conf. Multimedia, 2014, pp. 1041–1044, doi:10.1145/2647868.2655045.
- [10] Z. Rafii et al., "The MUSDB18 corpus for music separation", Zenodo, 2017, doi:10.5281/zenodo.1117372.
- [11] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks", Proc. 13th Int. Conf. Artificial Intelligence and Statistics, 2010, pp. 249–256.

Appendice

A. Funzioni olomorfe

Sia $f : \Omega \subseteq \mathbb{C} \rightarrow \mathbb{C}$, e $z \in \Omega$ con

$$z = x + iy \mapsto f(z) = u(x, y) + iv(x, y) = u(z) + iv(z)$$

dove $u(x, y)$ e $v(x, y)$ sono due funzioni reali con due variabili reali.

f è olomorfa, o differenziabile in senso complesso, nel suo dominio Ω se per ogni numero complesso z_0 esiste finito il seguente limite:

$$f'(z_0) = \lim_{\Delta z \rightarrow 0} \left[\frac{f(z_0 + \Delta z) - f(z_0)}{\Delta z} \right] \quad (29)$$

In tal caso $f \in H(\Omega)$.

Condizioni di Cauchy-Riemann

Δz può essere espresso come $\Delta x + i\Delta y$, mentre $f(z_0 + \Delta z) - f(z_0) = \Delta u(x, y) + i\Delta v(x, y)$, dunque:

$$f'(z_0) = \lim_{\Delta x \rightarrow 0} \lim_{\Delta y \rightarrow 0} \left[\frac{\Delta u(x, y) + i\Delta v(x, y)}{\Delta x + i\Delta y} \right] \quad (30)$$

Considerando il limite su due variabili, le direzioni verso il punto $(0, 0)$ nel piano complesso sono numerose e, per esistere, il valore deve essere lo stesso indipendentemente dalla direzione di arrivo. Le condizioni necessarie e sufficienti da verificare per la differenziabilità in senso complesso sono quelle di Cauchy-Riemann e si ottengono considerando i due limiti su un'unica direzione:

$$f'(z_0) = \begin{cases} \lim_{\Delta x \rightarrow 0} \left[\frac{\Delta u(x, y) + i\Delta v(x, y)}{\Delta x + i0} \right] \\ \lim_{\Delta y \rightarrow 0} \left[\frac{\Delta u(x, y) + i\Delta v(x, y)}{0 + i\Delta y} \right] \end{cases} \quad (31)$$

Considerando l'equazione corrispondente:

$$\frac{\partial u}{\partial x}(x_0, y_0) + i \frac{\partial v}{\partial x}(x_0, y_0) = -i \frac{\partial u}{\partial y}(x_0, y_0) + \frac{\partial v}{\partial y}(x_0, y_0) \quad (32)$$

si ottengono le condizioni di Cauchy-Riemann, che garantiscono la differenziabilità in senso complesso nel punto z_0 .

$$\begin{cases} \frac{\partial u}{\partial x}(x_0, y_0) = \frac{\partial v}{\partial y}(x_0, y_0) \\ -\frac{\partial u}{\partial y}(x_0, y_0) = \frac{\partial v}{\partial x}(x_0, y_0) \end{cases} \quad (33)$$

B. Efficienza computazionale con funzioni olomorfe

Sia $f : \Omega \subseteq \mathbb{C} \rightarrow \mathbb{C}$, $f \in H(\Omega)$, dove f è una funzione di attivazione. Considerando un generico algoritmo di ottimizzazione, come il Gradient Descent, è necessario calcolare il gradiente della funzione di costo e dunque delle funzioni di loss. Valutiamo la derivata della funzione di loss rispetto a un peso generico $w_{R,i}$ (in questo caso è stato scelto reale, ma non ci sarebbe alcuna differenza se venisse scelto immaginario).

$$\frac{dLoss}{dw_{R,i}} = \frac{dLoss}{dAct_1} \cdot \frac{dAct_1}{dAct_2} \cdot \frac{dAct_2}{dAct_3} \cdot \dots \cdot \frac{dAct_j}{dw_{R,i}} \quad (34)$$

dove:

$$\frac{dLoss}{dAct_1} = \left(\frac{dLoss}{dAct_1(1)} \quad \frac{dLoss}{dAct_1(2)} \quad \dots \quad \frac{dLoss}{dAct_1(n^{[L]})} \right) \in \mathbb{R}^{1, n^{[L]}} \quad (35)$$

$$\frac{dAct_j}{dw_{R,i}} = \begin{pmatrix} \frac{dAct_j(1)}{dw_{R,i}} \\ \vdots \\ \frac{dAct_j(n^{[L-j]})}{dw_{R,i}} \end{pmatrix} \in \mathbb{R}^{n^{[L-j]}} \quad (36)$$

$$\frac{dAct_r}{dAct_k} = \begin{pmatrix} \frac{dAct_r(1)}{dAct_k(1)} & \dots & \frac{dAct_r(1)}{dAct_k(n^{[L-k]})} \\ \frac{dAct_r(2)}{dAct_k(1)} & \dots & \frac{dAct_r(2)}{dAct_k(n^{[L-k]})} \\ \vdots & \ddots & \vdots \\ \frac{dAct_r(n^{[L-r]})}{dAct_k(1)} & \dots & \frac{dAct_r(n^{[L-r]})}{dAct_k(n^{[L-k]})} \end{pmatrix} \in \mathbb{R}^{n^{[L-r]}, n^{[L-k]}} \quad (37)$$

Considerando che ad ogni layer ci sono sia canali con numeri reali che canali con numeri immaginari, in egual misura (vedi sezione 4.3), l'insieme di essi è un insieme di numeri complessi. Riprendendo l'appendice A, una funzione complessa di variabile complessa ha in input due numeri, uno reale e uno immaginario, e ha in output altri due numeri, anch'essi rispettivamente reale e immaginario.

$$f(z_i) = u(z_i) + iv(z_i) \quad (38)$$

Consideriamo z_i come prodotto di una convoluzione (vedi sezione 4.3). Esso dipende linearmente da una serie di termini, risultati a loro volta di funzioni di attivazione, ma del layer precedente. Per ogni numero complesso del layer precedente che produce un numero complesso del layer successivo, sono dunque presenti 4 gradienti nella jacobiana tra la funzione di attivazione successiva e quella precedente. Consideriamo ad esempio il numero complesso $x_j + iy_j$, risultato della funzione di attivazione del layer precedente. Nel layer successivo ci sarà almeno un numero complesso che dipende da $x_j + iy_j$. Dunque, le derivate parziali $\frac{\partial u}{\partial x_j}, \frac{\partial u}{\partial y_j}, \frac{\partial v}{\partial x_j}, \frac{\partial v}{\partial y_j}$ sono elementi presenti nella Jacobiana descritta precedentemente nel calcolo di $\frac{dLoss}{dw_{R,i}}$. Come descritto nell'appendice A, se una funzione è olomorfa, allora valgono le condizioni di Cauchy-Riemann, descritte dalle equazioni 33. Dunque è possibile concludere affermando che l'utilizzo di funzioni di attivazione olomorfe consente di calcolare la metà delle derivate durante la backpropagation.